

The Heap Data Structure

? Prompt 1

What is a heap data structure?

✓ Answer

A **heap** is a **binary tree** that stores items (key–value pairs) and satisfies **two properties**:

1. **Heap-Order Property** (relational)
2. **Complete Binary Tree Property** (structural)

Heaps are mainly used to **implement priority queues efficiently**.

? Prompt 2

What is the heap-order property?

✓ Answer

In a **min-heap**:

For every node p (except the root),

$\text{key}(p) \geq \text{key}(\text{parent}(p))$

This means:

- The **minimum key is always at the root**
- Keys **increase (or stay equal)** as we move downward

? Prompt 3

What is the consequence of the heap-order property?

✓ Answer

- The smallest key is always at the **root**
- $\text{min}() \rightarrow \mathbf{O(1)}$ time
- Keys along any root-to-leaf path are in **nondecreasing order**

★ This is why heaps are ideal for **priority queues**.

◆ Min-Heap vs Max-Heap

? Prompt 4

Is this heap a min-heap or max-heap?

✓ Answer

This definition describes a **min-heap**.

Heap Type	Rule
Min-Heap	Parent $\leq$ Children
Max-Heap	Parent $\geq$ Children

◆ Important Clarification

? Prompt 5

Is a heap the same as the memory heap?

✓ Answer

✗ No.

- **Heap data structure** → Used in algorithms (priority queue)
- **Memory heap** → Runtime memory management

They are **completely unrelated** concepts.

◆ Why Structure Matters: Tree Height

? Prompt 6

Why do we want the heap to have small height?

✓ Answer

Operations like:

- insert
- remove-min
- heapify

take time proportional to the **height of the tree**.

☞ Smaller height = **faster operations**

That's why heaps enforce a **complete binary tree** structure.

◆ Complete Binary Tree Property (Structural Property)

? Prompt 7

What is a complete binary tree?

✓ Answer

A binary tree is **complete** if:

1. All levels except possibly the last are **completely full**
2. Nodes in the last level are filled **from left to right**

? Prompt 8

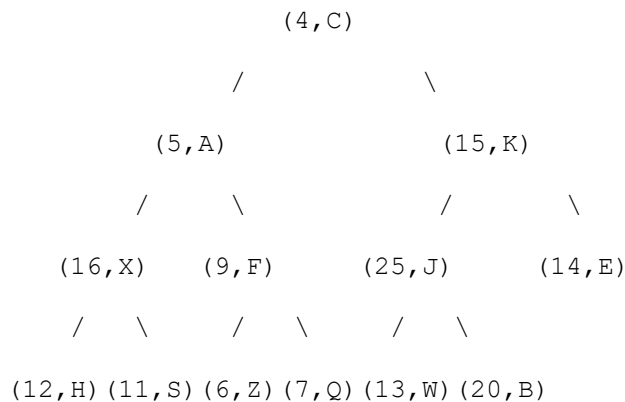
Why must a heap be complete?

✓ Answer

- Guarantees **height = $O(\log n)$**
- Enables **array-based representation**
- Ensures efficient priority queue operations

◆ Visual Example of a Heap (Figure 9.1)

♣ Heap Tree (Min-Heap)



- ✓ Heap-order property holds
- ✓ Complete binary tree
- ✓ Minimum element $(4, C)$ is at the root

◆ Why the Tree Is Complete

? Prompt 9

Why is this tree considered complete?

✓ Answer

- Levels 0, 1, and 2 are **completely full**
- Level 3 nodes occupy the **leftmost positions**
- No gaps appear before the last node

◆ Array-Based Representation of a Heap

? Prompt 10

How is a heap stored in an array?

✓ Answer

Because a heap is complete, it can be stored efficiently in an array:

```
Index:  0   1   2   3   4   5   6   7   8   9  10  11  12
Array: (4) (5) (15) (16) (9) (25) (14) (12) (11) (6) (7) (13) (20)
```

✦ Rules:

- Parent index = $\lfloor (i - 1) / 2 \rfloor$
- Left child = $2i + 1$
- Right child = $2i + 2$

◆ Last Position in a Heap

? Prompt 11

What is the “last position” in a heap?

✓ Answer

The **last position** is:

- The **rightmost node** on the **lowest level**
- The final array index ($n - 1$)

In Figure 9.1:

Last position $\rightarrow (13, w)$

This matters during **insert** and **remove-min** operations.

◆ Why Heaps Are Perfect for Priority Queues

? Prompt 12

Why are heaps ideal for implementing priority queues?

✓ Answer

Operation	Time
min()	$O(1)$
insert	$O(\log n)$
remove-min	$O(\log n)$

Thanks to:

- Heap-order property
- Complete binary tree property

Height of a Heap

? Prompt 1

What does Proposition 9.2 state?

✓ Answer

Proposition 9.2:

A heap storing n entries has height:

$$h = \lfloor \log n \rfloor$$

This means:

- The height of a heap grows **logarithmically**
- Heap operations are **efficient**

◆ Key Requirement

? Prompt 2

Why does this result depend on the heap being complete?

✓ Answer

Because a **complete binary tree**:

- Is filled level by level
- Has the **maximum number of nodes** for a given height

This allows us to tightly **bound the number of nodes** using powers of 2.

◆ Understanding Levels in a Heap

? Prompt 3

How many nodes are there at each level of a binary tree?

✓ Answer

Level	Number of nodes
0	$2^0 = 1$
1	$2^1 = 2$

2	$2^2 = 4$
...	...
i	2^i

◆ Counting Nodes in a Heap

? Prompt 4

How many nodes are there from level 0 to level h-1?

✓ Answer

This is a geometric series:

$$1 + 2 + 4 + \dots + 2^{h-1} = 2^h - 1$$

? Prompt 5

How many nodes can exist at level h (last level)?

✓ Answer

Because the tree is complete:

- Minimum: **1 node**
- Maximum: **2^h nodes**

◆ Bounding the Number of Nodes (Very Important)

? Prompt 6

What is the minimum number of nodes in a heap of height h?

✓ Answer

$$n \geq (2^h - 1) + 1 = 2^h$$

? Prompt 7

What is the maximum number of nodes in a heap of height h?

✓ Answer

$$n \leq (2^h - 1) + 2^h = 2^{h+1} - 1$$

◆ Inequalities for n

Now we have:

$$2^h \leq n \leq 2^{h+1} - 1$$

◆ Taking Logarithms

? Prompt 8

What happens when we take log on both sides?

✓ Answer

From the left inequality:

$$2^h \leq n \Rightarrow h \leq \log n$$

From the right inequality:

$$n \leq 2^{h+1} - 1 \Rightarrow \log((n+1)-1) \leq h$$

? Prompt 9

What do these inequalities tell us about h?

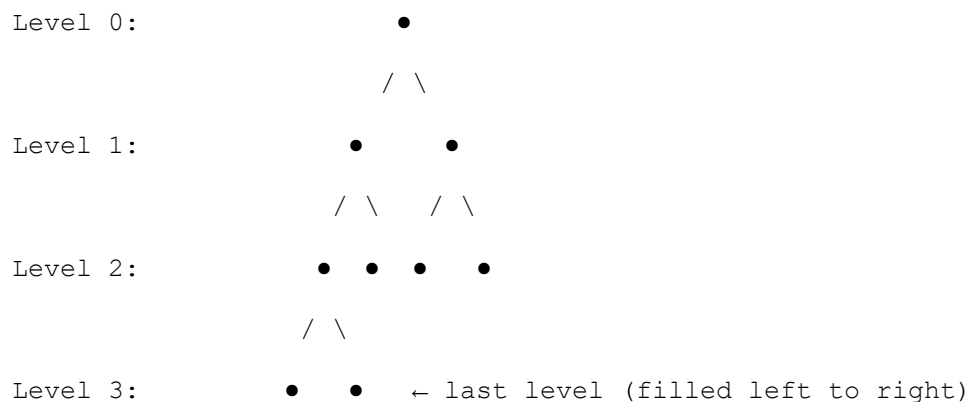
✓ Answer

$$\log(n+1)-1 \leq h \leq \log n$$

Since h is an **integer**, this implies:

$$h = \lfloor \log n \rfloor$$

♣ Visual Diagram (Heap Height)



- Height = number of levels – 1
- Nodes grow **exponentially**
- Height grows **logarithmically**

Adding an Item to a Heap (Priority Queue)

We assume a **min-heap** (smaller key = higher priority).

◆ PART A: Heap Basics (Quick Recall)

? Prompt

What is a heap in a priority queue?

✓ Answer

A heap is a **complete binary tree** that satisfies the **heap-order property**:

- **Complete Binary Tree Property**
 - All levels full except possibly the last
 - Last level filled **left to right**
- **Heap-Order Property (Min-Heap)**
 - Key at each node $\leq$ keys of its children

◆ PART B: Where Is a New Item Inserted?

? Prompt

Where is a new (k, v) inserted in a heap?

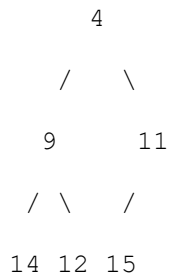
✓ Answer

The new item is placed:

- At the **next available position** at the bottom level
- Or as the **leftmost node of a new level** if the tree is full

This ensures the heap remains a **complete binary tree**.

Diagram: Placement of New Node



Insert new item here → next open spot

? Prompt

Why might heap-order be violated after insertion?

✓ Answer

Because the new key may be **smaller than its parent**, violating:

$\text{parent} \leq \text{child}$ ✗

So we must **restore heap order**.

◆ PART D: Up-Heap Bubbling (Key Concept)

? Prompt

What is up-heap bubbling?

✓ Answer

Up-heap bubbling is the process of:

- Comparing a node with its parent
- Swapping if the node's key is smaller
- Repeating until heap-order is restored

? Prompt

When does up-heap bubbling terminate?

✓ Answer

It stops when:

- The node becomes the **root**, or
- Its key $\geq$ parent's key

◆ PART E: Step-by-Step Algorithm (`add(k, v)`)

? Prompt

Explain the algorithm for `add(k, v)` in a heap.

✓ Answer

Step 1: Insert at correct position

- Place (k, v) at the next available position

Step 2: Compare with parent

- If $k \geq \text{parent.key} \rightarrow \text{stop}$
- Else $\rightarrow$ swap with parent

Step 3: Repeat upward

- Continue swapping until heap-order is restored

🌀 Pseudocode

`add(k, v) :`

`place (k, v) at next available leaf`

`p = position of new node`

`while p is not root:`

`q = parent(p)`

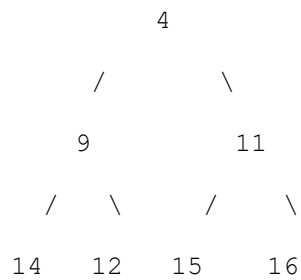
```

    if key(p) ≥ key(q):
        break
    swap(p, q)
    p = q

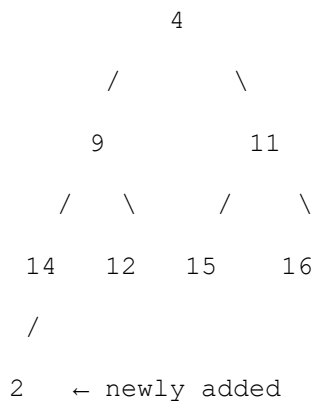
```

◆ PART F: Worked Example (Key = 2)

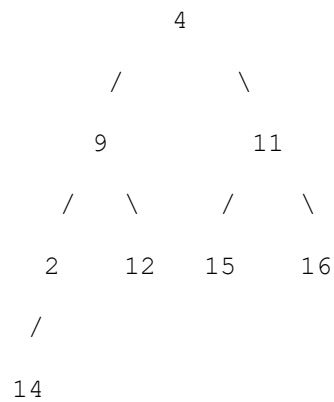
Initial Heap (Figure 9.2a)



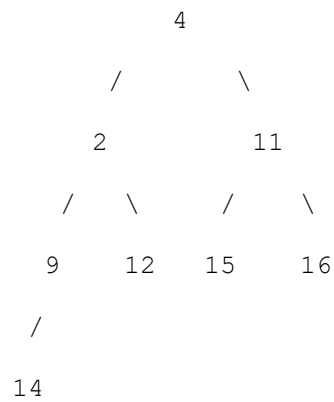
Step 1: Insert (2, T) (Figure 9.2b)



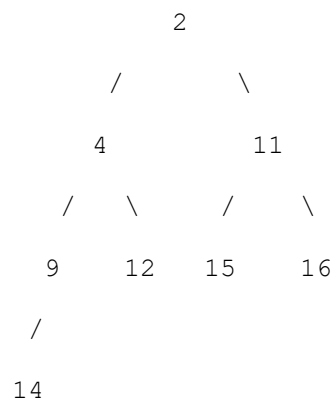
Step 2: Compare with parent (14) → swap (Figure 9.2c,d)



Step 3: Compare with parent (9) → swap (Figure 9.2e,f)



Step 4: Compare with parent (4) → swap (Figure 9.2g,h)



- ✓ Heap-order restored
- ✓ Bubbling stops at root

◆ PART G: Why Is It Efficient?

? Prompt

What is the time complexity of `add` in a heap?

✓ Answer

- Each swap moves the node up **one level**
- Maximum swaps = height of heap
- Height of a complete binary tree = **$\log n$**

🕒 Time Complexity

$O(\log n)$

Removing the Item with Minimum Key (`removeMin`)

We assume a **min-heap**.

◆ PART A: What Does `removeMin` Do?

? Prompt

What does the `removeMin` operation do in a priority queue?

✓ Answer

`removeMin`:

- Removes **and returns** the entry with the **smallest key**
- In a min-heap, this entry is always at the **root**

◆ PART B: Why We Can't Just Delete the Root

? Prompt

Why can't we directly delete the root of a heap?

✓ Answer

Deleting the root directly would:

- Break the **complete binary tree** property
- Leave **two disconnected subtrees**

Therefore, we must **replace** the root before deleting a node.

◆ PART C: Removing the Last Node

? Prompt

Which node is removed during removeMin?

✓ Answer

We remove the node at:

- The **last position p**
- The **rightmost node at the bottom level**

This preserves the **complete binary tree shape**.

? Prompt

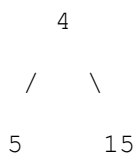
What happens to the item stored at the last node?

✓ Answer

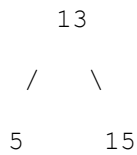
- The item at the last position is **copied to the root**
- The last node is then **deleted**

Diagram: Root Replacement

Before



After copying last node to root



(Then the last node is removed)

◆ PART D: Why Heap Order Is Violated

? Prompt

Why does replacing the root violate heap-order?

✓ Answer

Because the item copied from the last node:

- May have a **larger key than its children**
- Violates:
- $\text{parent} \leq \text{children}$
-

So we must **restore heap-order**.

◆ PART E: Down-Heap Bubbling (Core Idea)

? Prompt

What is down-heap bubbling?

✓ Answer

Down-heap bubbling:

- Moves an entry **downward** in the heap
- By swapping it with its **smaller child**
- Until heap-order is restored

? Prompt

Which child do we compare during down-heap bubbling?

✓ Answer

- If node has **only left child** → choose left child
- If node has **two children** → choose the child with the **smaller key**

This ensures heap-order is restored correctly.

◆ PART F: Step-by-Step Algorithm (**removeMin**)

? Prompt

Explain the algorithm for **removeMin** in a heap.

✓ Answer

Step 1: Save minimum

- Store root's element (this is the answer)

Step 2: Replace root

- Copy last node's item to root
- Delete last node

Step 3: Down-heap bubbling

- Compare root with smallest child
- Swap if root key > child key
- Repeat until heap-order is satisfied

🧠 Pseudocode

```
removeMin():
```

```
    if heap has one node:
```

```
        remove and return it
```

```
    save root element
```

```
    move last element to root
```

```
    delete last node
```

```
    p = root
```

```

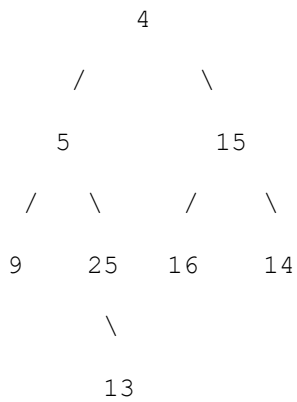
while p has a child:
    c = child of p with smaller key
    if key(p) ≤ key(c):
        break
    swap(p, c)
    p = c

return saved element

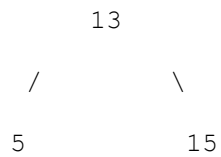
```

◆ PART G: Worked Example (Figure 9.3)

Initial Heap (Figure 9.3a)

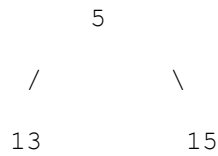


Step 1: Replace Root (Figure 9.3b)

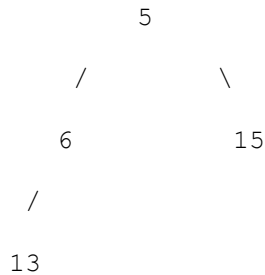


(last node removed)

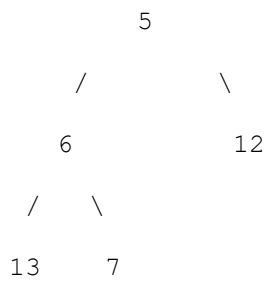
Step 2: Compare with Smaller Child (Figure 9.3c,d)



Step 3: Continue Bubbling (Figure 9.3e-f)



Step 4: Final Swap (Figure 9.3g-h)



- ✓ Heap-order restored
- ✓ Operation complete

◆ PART H: Time Complexity

? Prompt

What is the time complexity of `removeMin` in a heap?

✓ Answer

- Each swap moves the element **down one level**
- Maximum swaps = height of heap

- Height of heap = $\log n$

⚙️ Time Complexity

$O(\log n)$

◆ PART I: Key Differences (Insert vs Remove)

🔄 Up-Heap vs Down-Heap

Operation	Direction	Name
add	Bottom → Root	Up-heap bubbling
removeMin	Root → Bottom	Down-heap bubbling

Array-Based Representation of a Complete Binary Tree (Heap)

◆ PART A: Why Use an Array?

? Prompt

Why is an array-based representation ideal for a complete binary tree?

✓ Answer

Because in a **complete binary tree**:

- Nodes are filled **level by level**
- There are **no gaps** in positions

This allows the tree to be stored in an array with:

- **Contiguous indices**
- **No wasted space**

◆ PART B: Index Mapping Rules (VERY IMPORTANT)

? Prompt

How do we map nodes of a binary tree to an array?

✓ Answer

Each position p in the tree is stored at index $f(p)$ in the array.

✦ Index Rules

Tree Position	Array Index
Root	0
Left child of index i	$2i + 1$
Right child of index i	$2i + 2$
Parent of index i	$(i - 1) // 2$

🧠 Memory Formula

$\text{left}(i) = 2i + 1$

$\text{right}(i) = 2i + 2$

$\text{parent}(i) = (i-1) // 2$

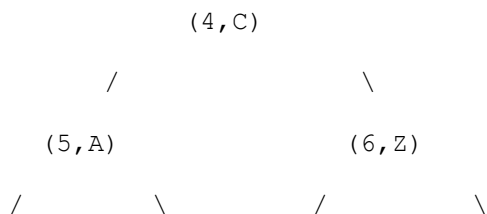
◆ PART C: Example Heap Stored in an Array

? Prompt

Illustrate an array-based representation of a heap.

✓ Answer

Tree View



(15, K) (9, F) (7, Q) (20, B)

Array View (Figure 9.4)

Index: 0 1 2 3 4 5 6

Value: (4, C) (5, A) (6, Z) (15, K) (9, F) (7, Q) (20, B)

- ✓ Root at index 0
- ✓ Last node always at index $n-1$

◆ PART D: Key Properties of Array-Based Heaps

? Prompt

Where is the last node of a heap stored in an array?

✓ Answer

If the heap has n elements:

Last position index = $n - 1$

This makes:

- Insertion easy
- Removal easy

◆ PART E: How Operations Become Easier

? Prompt

Why is insertion simpler with an array-based heap?

✓ Answer

Because:

- New elements are placed at index n
- Parent index is computed directly using $(i-1) // 2$
- No tree traversal is required

? Prompt

Why is `removeMin` easier using an array-based heap?

✓ Answer

Because:

- Root is always at index 0
- Last element is always at index $n-1$
- Replacement and bubbling use index arithmetic only

◆ PART F: Up-Heap and Down-Heap Using Indices

▲ Up-Heap Bubbling (Insertion)

```
while i > 0 and A[i] < A[parent(i)]:  
    swap A[i] and A[parent(i)]  
    i = parent(i)
```

▼ Down-Heap Bubbling (Removal)

```
while left(i) exists:  
    smaller = index of smaller child  
    if A[i] ≤ A[smaller]:  
        break  
    swap A[i] and A[smaller]  
    i = smaller
```

◆ PART G: Space and Time Complexity

? Prompt

What is the space complexity of an array-based heap?

✓ Answer

The array stores all n elements:

$O(n)$

? Prompt

What are the time complexities of heap operations?

✓ Answer

Operation	Time Complexity
add	$O(\log n)$ (amortized)
removeMin	$O(\log n)$ (amortized)
min	$O(1)$

◆ PART H: Dynamic Resizing (Important Note)

? Prompt

What happens if heap size exceeds array capacity?

✓ Answer

- The array is **resized dynamically**
- Usually doubled in size
- Causes **amortized time bounds**

This is similar to Python's `list`.

◆ PART I: Array-Based vs Linked-Tree Heaps

Comparison

Feature	Array-Based Heap	Linked Tree Heap
Parent/child access	$O(1)$	Requires pointers
Last node access	$O(1)$	Hard
Space efficiency	High	Lower
Implementation	Simple	Complex

Bottom-Up Heap Construction (Linear-Time Heapify)

◆ PART A: Motivation

? Prompt

What is the time complexity of building a heap using repeated add operations?

✓ Answer

If we insert n elements one by one:

- Each `add` may take $O(\log n)$
- Total time:

$O(n \log n)$

? Prompt

When is bottom-up heap construction used?

✓ Answer

When **all n key–value pairs are known in advance**, such as:

- First phase of **Heap Sort**
- Constructing a heap from an existing array

In this case, we can build the heap in:

$O(n)$ time

◆ PART B: Key Idea of Bottom-Up Construction

? Prompt

What is bottom-up heap construction?

✓ Answer

Bottom-up heap construction:

- Starts from the **lowest non-leaf nodes**
- Applies **down-heap (sink)** operations
- Ends at the **root**
- Fixes heap order **locally**, not globally

🔑 Core Insight

Leaves are already heaps of size 1, so we only need to fix **internal nodes**.

◆ PART C: Conceptual Step-by-Step Construction

(Assume $n = 2^{(h+1)} - 1$, a perfect binary tree)

? Prompt

Explain bottom-up heap construction step by step.

✓ Answer

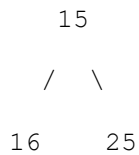
◆ Step 1: 1-Entry Heaps (Leaves)

Each leaf node is already a valid heap.

[16] [25] [9] [15] [4] [12] [11] [6]

◆ Step 2: Build 3-Entry Heaps

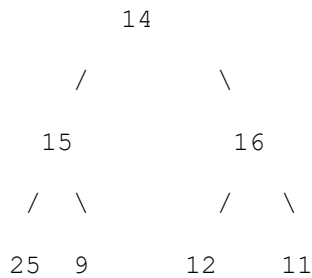
Combine pairs of 1-entry heaps and add a parent:



Apply **down-heap** if parent violates heap order.

◆ Step 3: Build 7-Entry Heaps

Join two 3-entry heaps and add a new root:



Down-heap bubbling may occur.

◆ Final Step: Build Full Heap

Join the last two large heaps and down-heap from root.

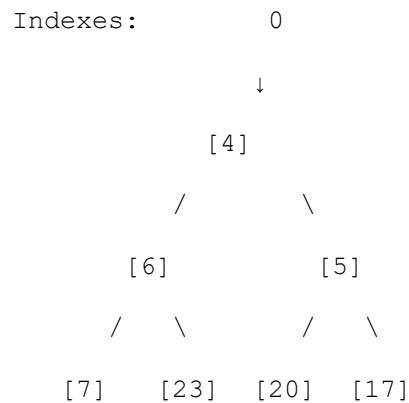
◆ PART D: Diagram (Exam-Friendly)

Example: Bottom-Up Construction (n = 15)

Initial Array (Arbitrary Order):

[16, 25, 9, 15, 4, 12, 11, 6, 17, 7, 23, 20, 14, 8, 5]

Heapify from last non-leaf:



Highlighted paths are **down-heap bubbling paths**.

◆ PART E: Why Is This $O(n)$ Time?

? Prompt

Why does bottom-up heap construction run in $O(n)$ time?

✓ Answer

Although a single down-heap can take $O(\log n)$:

- Most nodes are near the **bottom**
- Bottom nodes move very little
- Only a few nodes (near root) move far

▮ Work Distribution

Node Level	# Nodes	Cost per Node
Leaves	$n/2$	0
Above leaves	$n/4$	1
Higher	$n/8$	2
Root	1	$\log n$

Total work sums to:

$O(n)$

◆ PART F: Array-Based Heapify Algorithm

? Prompt

How do we implement bottom-up heap construction using an array?

✓ Answer

Steps:

1. Store all elements in array (any order)
2. Find the **parent of last index**
3. Call `downheap()` from that index to the root

🔑 Key Formula

```
start = parent(n - 1)
```

◆ PART G: Python Implementation (Conceptual)

Constructor with heapify

```
def __init__(self, contents=()):  
    self._data = [self.Item(k, v) for k, v in contents]  
    if len(self._data) > 1:  
        self._heapify()
```

Heapify Method

```
def _heapify(self):  
    start = self._parent(len(self._data) - 1)  
    for j in range(start, -1, -1):  
        self._downheap(j)
```

◆ PART H: Why Start from the Last Non-Leaf?

? Prompt

Why does heapify start at the deepest non-leaf node?

✓ Answer

Because:

- Leaves have no children
- Down-heap does nothing on leaves
- Saves unnecessary operations

◆ PART I: Heap Sort Connection

? Prompt

If heapify is $O(n)$, why is heap sort still $\Theta(n \log n)$?

✓ Answer

Heap sort has two phases:

Phase	Time
Build heap	$O(n)$
Remove min/max n times	$\Theta(n \log n)$

Overall:

$\Theta(n \log n)$